

HealthVault Trust Layer: Complete Build Plan v2 (MST Buildathon)

0. What changed since v1

Area	v1	v2
Onboarding	SARAL for workers and patients	Dropped as a dependency. No developer docs found. Relay creates and holds worker keys (custodial for the pilot, stated honestly)
Signing	Backend relayer signs everything	Split: admin and hospital sign with BridgeKey on a hosted web app. Workers are signed by the relay
Screens	Admin and hospital screens inside the APK	Admin and hospital move to a hosted web app. The APK keeps the worker flow
Privacy	Klethesia ZK consent	Hash + access mapping. No ZK claim. WASMify not needed
Tooling	Raw web3.py + Hardhat	Python SDK (deploy, balances, receipts, token sends) + web3.py for our contract calls + Hardhat to compile. Vibe Kit or Next.js + wagmi for the web app
Chain details	Unconfirmed	RPC URLs confirmed, chain ID likely (verify), faucet, explorer, wallet confirmed
Delivery	Local demo	Hosted relay + web app, public APK link, public repo + README, compulsory submission form
Extras	None	Scale story in the pitch, optional Social Award video

Unchanged: the five contracts, hashing rules, consent expiry, tamper check, AI-decision anchor, CareCoin stipend flow, the three roles, and the rule that only hashes, permissions and payouts go on-chain.

1. What we are building

HealthVault is the existing ASHA app (ASHA worker, govt hospital, admin) plus a blockchain trust layer on MST:

- **Identity:** who is acting (workers, hospitals, families)
- **Validation:** is a record real and unaltered, and did the AI output change after it was made
- **Consent:** who may see which data, for how long (auto-expiring)
- **Payout:** a hospital-attested visit releases CareCoin to the worker automatically

Names, Aadhaar, vitals and history stay in local storage. Tracks: Identity & Verification (primary), Enterprise/Government/Social Impact, Consumer Blockchain & Payments, Blockchain & Web3.

Pitch in one line: one system that verifies both people and predictions, and pays the frontline worker once a visit is proven.

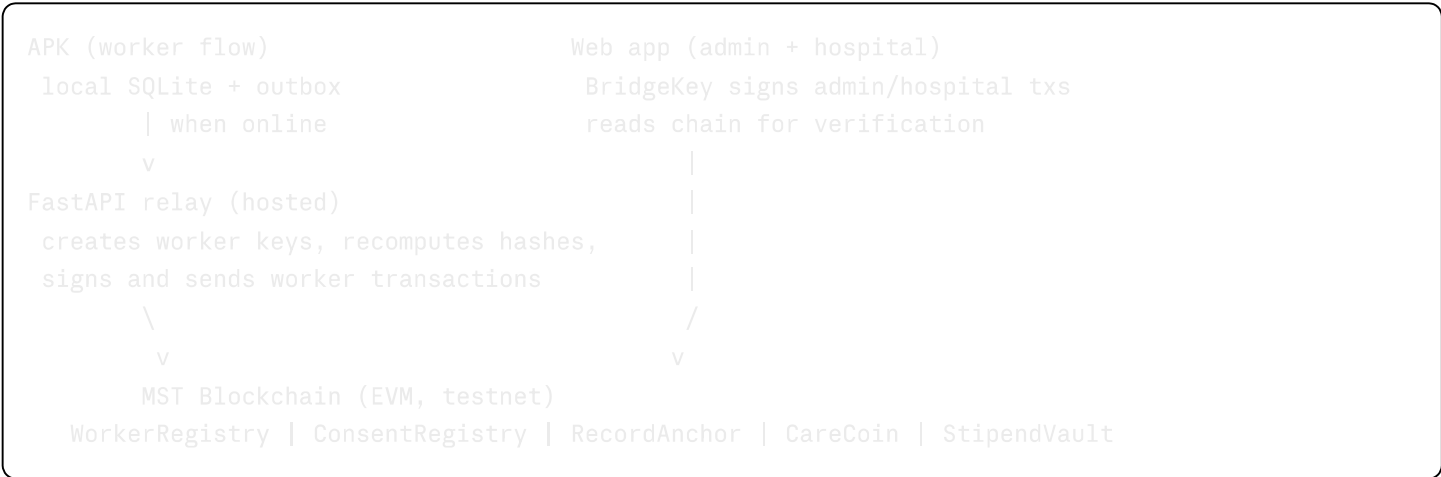
2. Confirmed vs verify (hour zero)

Item	Status	Action
Testnet RPC (<code>https://testnetrpc.mstblockchain.com</code>), mainnet (<code>https://mariorpc.mstblockchain.com/</code>)	Confirmed (Python SDK page)	Use
Chain ID 91562037 (testnet), 4646 (mainnet)	From a Vibe Kit pull request	Confirm with <code>eth_chainId</code>
Faucet, MSTScan, BridgeKey, Python and TS SDKs	Confirmed (guide)	Use
Solidity 0.8.24 / Cancun EVM	From a paste	Test-deploy a tiny contract. If opcode errors, lower <code>evmVersion</code>
BridgeKey from a Flutter APK	Unknown	Avoid depending on it. Web app is the BridgeKey surface
SARAL, Klethesia, WASMify developer access	No docs found	Not needed. Mention only if actually used

3. Roles and where each lives

- **Admin (web app, BridgeKey):** register/revoke workers, authorize hospitals, fund the vault, set rates.
- **Hospital (web app, BridgeKey):** verify QR, log access, attest visits.
- **ASHA worker (APK, relay-signed):** record visits offline, capture consent, generate QR, earn CareCoin.

4. Architecture



5. What goes where

On-chain	Off-chain
Worker credential hash, revocation	Names, Aadhaar, phone numbers
Consent grants, categories, expiry	Vitals, history, family details
Record Merkle roots, AI digests	The records and the model
Visit keys, payouts, CareCoin balances	Beneficiary details, family salts

6. Features

A. Identity. Admin registers a worker: the relay creates the worker's key, the admin signs `register(worker, credentialHash)`. Family head gets a root identity, members `memberId = keccak256(familyRoot, memberIndex)`. Remove worker fires `revoke()`.

B. Consent. Worker's app captures categories, provider and duration. Stored on-chain with expiry, so access ends on its own. Access log built from events. Patient is a salted commitment, never a raw ID.

C. Integrity and AI provenance. Canonicalize each visit's vitals, hash, Merkle-root a batch, anchor it. When the sepsis/NEWS2 model flags high risk, anchor `aiDigest = keccak256(modelVersion, inputHash, outputLabel)`. The referral QR carries the record payload. The web verifier recomputes the hash and compares it to the chain. Editing one vital makes the check fail visibly.

D. Stipend (CareCoin). Worker submits `visitKey = keccak256(beneficiaryCommitment, taskType, period)`. Hospital calls `attestAndPay`. The same visit cannot be paid twice. CareCoin is test money and a settlement rail that could trigger real payouts such as DBT.

E. Verifier screen (web, hospital). Paste or scan the QR payload, then one screen shows worker identity, consent, record integrity and visit status.

F. Bonus after A to E: portability across ASHA areas, hashed duplicate-beneficiary check, logged emergency access.

7. Contracts (Solidity ^0.8.24, sketches to adapt and test)

```
solidity
```

```
// IReg.sol
interface IReg { function isActive(address) external view returns (bool); }
```

```
solidity
```

```
// WorkerRegistry.sol
contract WorkerRegistry {
    address public admin;
    mapping(address => bytes32) public credential;
    mapping(address => bool) public active;
    event Registered(address indexed w, bytes32 credHash);
    event Revoked(address indexed w);
    modifier onlyAdmin() { require(msg.sender == admin, "admin only"); _; }
    constructor() { admin = msg.sender; }
    function register(address w, bytes32 h) external onlyAdmin { credential[w] = h; active[w] = true; }
    function revoke(address w) external onlyAdmin { active[w] = false; emit Revoked(w); }
    function isActive(address w) external view returns (bool) { return active[w]; }
}
```

solidity

```
// ConsentRegistry.sol
import "./IReg.sol";
contract ConsentRegistry {
    IReg public reg;
    struct Consent { bytes32 patient; address provider; uint16 categories; uint64 expiry; bool revoked; }
    Consent[] public consents;
    event Granted(uint256 indexed id, bytes32 indexed patient, address indexed provider, uint16 categories);
    event Revoked(uint256 indexed id);
    event Accessed(uint256 indexed id, address indexed by, uint16 category);
    constructor(address r) { reg = IReg(r); }
    function grant(bytes32 patient, address provider, uint16 cats, uint64 expiry) external returns (uint256) {
        require(reg.isActive(msg.sender), "worker not active");
        require(expiry > block.timestamp, "bad expiry");
        consents.push(Consent(patient, provider, cats, expiry, false));
        id = consents.length - 1;
        emit Granted(id, patient, provider, cats, expiry);
    }
    function revoke(uint256 id) external { require(reg.isActive(msg.sender), "worker not active"); }
    function isAllowed(uint256 id, address who, uint16 cat) public view returns (bool) {
        Consent storage c = consents[id];
        return !c.revoked && c.provider == who && block.timestamp < c.expiry && (c.categories & cat == cat);
    }
    function logAccess(uint256 id, uint16 cat) external { require(isAllowed(id, msg.sender, cat), "not allowed"); }
}
```

Consent is submitted by the worker's app on the patient's behalf. Patient-signed consent is a stretch goal, and you should say so.

solidity

```
// RecordAnchor.sol
import "./IReg.sol";
contract RecordAnchor {
    IReg public reg;
    struct A { address by; uint64 at; bytes32 aiDigest; }
    mapping(bytes32 => A) public anchors;
    event Anchored(bytes32 indexed root, address indexed by, bytes32 aiDigest);
    constructor(address r) { reg = IReg(r); }
    function anchor(bytes32 root, bytes32 aiDigest) external {
        require(reg.isActive(msg.sender), "worker not active");
        require(anchors[root].at == 0, "already anchored");
        anchors[root] = A(msg.sender, uint64(block.timestamp), aiDigest);
        emit Anchored(root, msg.sender, aiDigest);
    }
    function exists(bytes32 root) external view returns (bool) { return anchors[root].at != 0; }
}
```

solidity

```
// CareCoin.sol (OpenZeppelin v5)
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract CareCoin is ERC20 { constructor() ERC20("CareCoin", "CARE") { _mint(msg.sender, 1_000_000
```

solidity

```
// StipendVault.sol
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "./IReg.sol";
contract StipendVault {
    address public admin; IReg public reg; IERC20 public token;
    mapping(address => bool) public hospitals;
    mapping(uint8 => uint256) public rate;
    struct Visit { address worker; uint8 taskType; bool paid; }
    mapping(bytes32 => Visit) public visits;
    event VisitSubmitted(bytes32 indexed key, address indexed worker, uint8 taskType);
    event StipendPaid(bytes32 indexed key, address indexed worker, uint256 amount);
    modifier onlyAdmin() { require(msg.sender == admin, "admin only"); _; }
    constructor(address r, address t) { admin = msg.sender; reg = IReg(r); token = IERC20(t); }
    function setHospital(address h, bool ok) external onlyAdmin { hospitals[h] = ok; }
    function setRate(uint8 t, uint256 amt) external onlyAdmin { rate[t] = amt; }
    function submitVisit(bytes32 key, uint8 taskType) external {
        require(reg.isActive(msg.sender), "worker not active");
        require(visits[key].worker == address(0), "duplicate visit");
        require(rate[taskType] > 0, "unknown task");
        visits[key] = Visit(msg.sender, taskType, false);
        emit VisitSubmitted(key, msg.sender, taskType);
    }
    function attestAndPay(bytes32 key) external {
        require(hospitals[msg.sender], "not a hospital");
        Visit storage v = visits[key];
        require(v.worker != address(0) && !v.paid, "invalid or paid");
        v.paid = true;
        require(token.transfer(v.worker, rate[v.taskType]), "transfer failed");
        emit StipendPaid(key, v.worker, rate[v.taskType]);
    }
}
```

Deploy order: WorkerRegistry → CareCoin → StipendVault(registry, token) → ConsentRegistry(registry) → RecordAnchor(registry). Then the admin transfers CareCoin into the vault, calls `setRate`, and `setHospital` for the hospital's BridgeKey address.

8. Wallets and signing

- **Admin and hospital:** BridgeKey in the browser, connected to MST Testnet. Judges see real wallet-signed transactions.
- **Workers:** the relay creates one key per worker (Python SDK `Client.create_random`), stores it encrypted server-side, and signs worker transactions. It also tops up each worker with a little \$MSTC for gas.
- **Say honestly:** worker keys are custodial in this pilot. A production version would move to non-custodial key management.
- **Never** commit private keys. Use environment variables and a `.env.example`. The guide prints shared MCP credentials, so don't reuse them.

9. Hosting and submission (mandatory)

- **Deployed on MST Testnet** with contract addresses recorded.
- **At least one real transaction hash** demonstrating core function. Every hash you show must be real, or you risk disqualification.
- **Hosted demo:** relay and web app on free-tier hosts, plus a public APK download link.
- **Public GitHub repo:** contracts, relay, web app, APK source. README covers MST integration, contract addresses and setup.
- **Submission form (compulsory):** repo, contract address, tx hash, demo link, demo video, all public. Form: <https://forms.gle/fkkVbfiwKmbL3BFp9>
- **Optional Social Award:** Instagram video of at least 30 seconds, faces on camera, tag @mstblockchain and @newrro_tech, say you're building on MST, and submit it through the form.
- **AI policy:** AI tools are allowed, but everyone must be able to explain the code, the architecture and the MST integration.

10. App, relay and web changes

APK: `chain_outbox` table `((id, kind, payload_json, status, tx_hash, created_at))`, a hashing module, a consent screen, a QR generator, and chain-status badges (pending, anchored, tampered).

Relay (FastAPI): `(POST /register-worker, /consent, /anchor, /visit)`, each recomputing hashes from the payload. Uses the Python SDK for deploy, balances and receipts, and web3.py for our contract calls, against `(https://testnetrpc.mstblockchain.com)`. Works through the outbox when the phone syncs.

Web app (Next.js + wagmi or Vibe Kit): admin panel, hospital verifier, live event feed with mstscan links.

Hashing rules (same in Dart, Python and JS): canonical JSON with fixed field order and number formats, then keccak256. Beneficiary commitment is `(keccak256(beneficiaryId + familySalt))`, with the salt kept on the device. Visit key excludes timestamp and GPS. Keep shared test vectors and test them first.

Hardhat config: Solidity 0.8.24, network URL from the RPC above, chain ID 91562037 (verify), accounts from env.

11. Build order

1. **Hour zero:** faucet, `(eth_chainId)` check, test-deploy a tiny contract and find it on mstscan.com, install BridgeKey, watch the deployment and BridgeKey tutorials.
2. **Contracts:** compile, test locally, deploy in the order above, record addresses.
3. **Relay and hashing:** one hash matching across Dart, Python and JS before anything else.
4. **Identity:** admin registers a worker (BridgeKey), revoke works.
5. **Consent:** grant, expiry, access log.
6. **Integrity and AI anchor:** anchor a visit, edit a vital, watch the check fail.
7. **Web verifier:** the one-screen result.
8. **CareCoin and stipend:** fund, attest, balance updates.
9. **Host everything,** publish the APK, write the README, and prepare synthetic data.
10. **Record the real end-to-end run,** cut the demo video (and the Social Award video), submit the form.
11. **Bonus features** only if the above is solid.

Stopping after step 7 still gives a complete entry. Step 8 is the finale.

12. Demo script (3 minutes)

Minute 1, problem. Stipends delayed for months, forged or duplicate records, patient data with no control.

Minute 2, live flow. Admin registers a worker (BridgeKey signature) → worker records a visit and the patient grants consent → hospital verifies the QR: identity valid, consent granted, record intact → hospital attests and CareCoin lands in the worker's balance → edit one vital and the tamper alarm fires → resubmit the same visit and it is rejected.

Minute 3, proof. Open the transaction on mstscan.com: the hash, the event, the balance change. Close on what stays private.

Scale line: a district-level pilot with an admin dashboard and a settlement rail toward DBT. Do not invent user numbers.

13. Honest framing

- The chain proves a record hasn't changed since anchoring and that the right party was authorized. It can't prove a visit happened. The hospital attestation covers that.
- CareCoin is test money and a settlement rail, not a stipend replacement.
- Worker keys are relay-held. Say so.
- Claim an MST protocol only if it actually runs.
- Synthetic data only, labelled as synthetic. Transactions shown must be real.

14. Risks and fallbacks

Risk	Fallback
Solidity/EVM version mismatch	Lower <code>evmVersion</code> and recompile
BridgeKey connection trouble	Test early. Fallback is signing admin/hospital transactions with a funded key via the SDK, and saying so
RPC or host flaky during demo	Video of a real run, plus existing transactions on the explorer
Hash mismatch across Dart/Python/JS	Shared test vectors, canonical JSON rules
Free-tier host sleeping	Warm it up before the demo
Offline sync edge cases	Demo online, show the outbox as a talking point
Too much scope	Stop after step 7 and polish